

三個檔案逐行講解 — counter.v / tb_counter.v / wave.vcd

這份筆記把「程式碼」和「GTKWave 上看到的波形圖」一行一行對起來。看不懂語法時，先回到最上面的語法急救表查。

目錄

- [0. 語法急救表](#)
 - [1. counter.v — 這個電路長什麼樣](#)
 - [2. tb_counter.v — 怎麼測試它](#)
 - [3. wave.vcd — 發生了什麼事](#)
 - [4. 三個檔案跟那張圖的關係](#)
 - [附註：一個你可能會問的細節](#)
-

0. 語法急救表

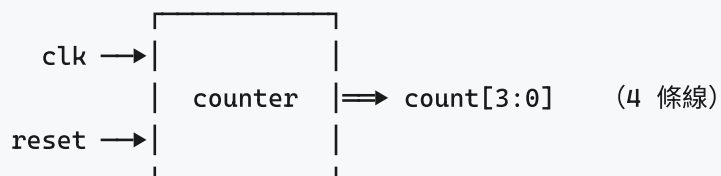
符號	意思
//	註解，後面的字電腦不看
;	一句話結束（最常忘記的錯誤）
begin / end	等於其他語言的大括號
[3:0]	4 條線，編號 3、2、1、0
4'b0000	4 位元的二進位值 0000
<=	不是小於等於！是「賦值」（時脈電路專用）
=	賦值（testbench 專用）
~	NOT，0 變 1、1 變 0
#5	等 5 個時間單位（只有測試檔能用）
\$ 開頭	給模擬器的指令，不是硬體
@(...)	「在.....發生的時候」

1. counter.v — 這個電路長什麼樣

第 1–5 行：定義腳位

```
module counter (           // 第1行
    input wire clk,         // 第2行
    input wire reset,       // 第3行
    output reg [3:0] count  // 第4行
);                          // 第5行
```

想像你在畫一顆 IC 的接腳圖：



這段字	意思
<code>module counter</code>	我要定義一個零件，型號叫 <code>counter</code>
括號裡的内容	這顆零件的 所有接腳
<code>input</code>	這隻腳是 進來的
<code>output</code>	這隻腳是 出去的
<code>wire</code>	這隻腳只是一條線，值由外面決定
<code>reg</code>	這隻腳的值要 被記住 （後面會存進正反器）
<code>[3:0]</code>	這不是一隻腳，是 4 隻腳綁在一起

[!WARNING] 最常見的錯：腳位之間要有逗號 `,`，但最後一個不能有。第 4 行 `count` 後面沒逗號，第 2、3 行有。

第 8 行：什麼時候動

```
always @(posedge clk) begin
```

拆成三塊唸：

<code>always</code>	<code>@(posedge clk)</code>	<code>begin</code>
「永遠」	「在 <code>clk</code> 上升緣時」	「開始做這些事」

`posedge` = **positive edge**（正緣），就是 `clk` 從 0 跳到 1 的**那一瞬間**。

[!IMPORTANT] 這一行就是波形圖上「`count` 為什麼只在 `clk` 往上跳的時候才變」的原因。你在 GTKWave 上看到的 `count` 階梯，每一階都對齊 `clk` 的上升邊——就是這行造成的。

第 9–12 行：動的時候做什麼

```
if (reset)
    count <= 4'b0000;    // reset 是 1 → 歸零
else
    count <= count + 1;  // 否則 → 加 1
```

`4'b0000` 拆開來看：

```
4 ' b 0000
| | | └─ 值
| | └── b = binary 二進位 (d=十進位, h=十六進位)
| └── 分隔符號，固定要寫
└── 總共 4 個位元
```

所以 `4'b0000`、`4'd0`、`4'h0` 是同一件事，只是寫法不同。

關於 `<=`：在 `always @(posedge clk)` 裡面，`<=` 不是比較大小，是賦值。它跟 `=` 的差別是「同時發生」vs「依序發生」。

[!TIP] 初學先記口訣就好：`always @(posedge clk)` 裡面 → 一律用 `<=`

最後一行

```
endmodule
```

零件定義結束。注意它後面沒有分號。

這 15 行變成什麼

用 Yosys 綜合過，它會變成：

```
4 個正反器 (記住 count 的 4 個位元)
4 個 LUT    (算出「下一個值是多少」)
2 個進位鏈  (處理加法的進位)
```

2. tb_counter.v — 怎麼測試它

[!NOTE] 這個檔完全不會變成硬體。它是「實驗室」，只存在於模擬裡。

第 1 行

```
module tb_counter;
```

注意：沒有括號、沒有腳位。因為它不接到任何外面的東西——它就是整個世界。

第 4–9 行：準備訊號

```
reg clk;           // 我要「產生」的
reg reset;         // 我要「產生」的
wire [3:0] count;  // 我要「接收」的
```

規則超簡單：

這條訊號是	用什麼	類比
我自己控制的	reg	我手上的開關
對方輸出給我的	wire	接上去的電線，值由對方決定

`clk` 和 `reset` 是我要餵給 counter 的，所以是 `reg`。`count` 是 counter 吐出來的，我只能看，所以是 `wire`。

第 12–16 行：把零件裝上去

```
counter uut (
    .clk(clk),
    .reset(reset),
    .count(count)
);
```

這叫**實例化** (instantiation)，就是「拿一顆 counter 出來，插到板子上」。

```
counter    uut    (...)
```

└──┬──┘

└──┬──┘ 我幫這一顆取的名字 (Unit Under Test，待測物)

└──┬──┘ 用哪個型號

括號裡每一行是在接線：

```
.clk ( clk )
```

└──┬──┘

└──┬──┘ 我這邊的線名

└──┬──┘ counter 那顆的腳位名

唸成：「把 counter 的 `clk` 腳，接到我的 `clk` 線上。」

就像插排線 —— 左邊是 IC 的腳，右邊是你的杜邦線。名字剛好一樣只是為了好記，其實可以不一樣。

第 19 行：產生時脈

```
always #5 clk = ~clk;
```

[!IMPORTANT] 這行直接對應你在圖上看到的黃色方波。

- `always` 後面沒有 `@(...)` → 就是「無限重複」
- `#5` → 等 5 個時間單位
- `~clk` → 把 `clk` 反過來

所以：等 5 → 翻轉 → 等 5 → 翻轉 → ...

```
時間： 0    5    10    15    20    25
clk：  0 → 1 → 0 → 1 → 0 → 1
      └─週期 = 10─┘
```

你在 GTKWave 上量到的 `clk` 週期是 10，就是這行 `#5` 來的。

第 22 行：initial 區塊

```
initial begin
```

`initial` = 模擬一開始執行一次，從上往下依序做。

跟 `always` 的差別：`always` 會一直重複，`initial` 只跑一遍。

第 24–25 行：開始錄影

```
$dumpfile("wave.vcd");
$dumppvars(0, tb_counter);
```

[!IMPORTANT] 這兩行就是 `wave.vcd` 存在的唯一原因。沒有它們，你什麼圖都看不到。

- `$dumpfile("wave.vcd")` → 錄影檔取名叫 wave.vcd
- `$dumpvars(0, tb_counter)` → 從 `tb_counter` 開始錄
 - 第一個參數 `0` = 錄所有層級，包含裡面的 `uut`
 - 這就是為什麼 GTKWave 左邊的樹會有 `tb_counter` 底下掛著 `uut`

第 28–34 行：測試劇本

```

    clk    = 0;           // t=0    時脈起始值
    reset  = 1;           // t=0    按住重置鈕

    #15 reset = 0;        // t=15   放開重置鈕

    #100;                 // 再跑 100 個單位 → t=115

    $finish;              // 結束模擬

```

畫成時間軸：

```

t=0  ————— t=15  ————— t=115
|           |           |
|           |           |
clk=0       reset 放開   $finish
reset=1
└─ 波形圖左邊紅線是高的那段 ─┘

```

波形圖總長度是 115，就是 `15 + 100` 算出來的。

[!NOTE] `#15` 是「再等 15」，不是「跳到第 15」。 `#100` 是從 15 再等 100 = 115。

這裡用 `=` 而不是 `<=`，因為 `initial` 裡面是「依序做」，跟一般寫程式一樣。

第 38–40 行：印文字

```

initial begin
    $monitor("時間=%0t | 重置=%b | 計數值=%b (%d)", $time, reset, count,
count);
end

```

這是在終端機看到那幾行字的來源，跟波形圖無關。

- `$monitor` = 只要參數裡任何一個變了，就自動印一行
- `%0t` = 時間、`%b` = 二進位、`%d` = 十進位
- `$time` = 現在的模擬時間

[!IMPORTANT] 重要觀念：這個檔有兩個 `initial` 區塊，它們是同時執行的，不是第一個做完才做第二個。Verilog 裡的區塊天生就是平行的——因為硬體本來就是同時在動。這是跟寫軟體最不一樣的地方。

3. wave.vcd — 發生了什麼事

這是 `vvp` 產生的錄影檔，純文字。分成兩半：前面是目錄，後面是流水帳。

第 1–9 行：檔頭

<code>\$date</code>	<code>Thu Aug 27 10:30:43 2026</code>	← 什麼時候錄的
<code>\$version</code>	<code>Icarus Verilog</code>	← 誰錄的
<code>\$timescale</code>	<code>1s</code>	← 一個時間單位 = 1 秒

[!NOTE] `1s` 是因為你的 `.v` 檔沒寫 `timescale` 指令，模擬器就預設 1 秒。所以 GTKWave 上顯示「115 sec」。真實硬體當然不會是秒，之後在檔案最上面加一行 ``timescale 1ns/1ps` 就會變成奈秒。

第 10–20 行：訊號目錄

<code>\$scope module tb_counter \$end</code>	← 進入 <code>tb_counter</code> 這一層
<code>\$var wire 4 ! count [3:0] \$end</code>	← 有個 4 位元的 <code>count</code> ，代號 <code>!</code>
<code>\$var reg 1 " clk \$end</code>	← 有個 1 位元的 <code>clk</code> ，代號 <code>"</code>
<code>\$var reg 1 # reset \$end</code>	← 有個 1 位元的 <code>reset</code> ，代號 <code>#</code>
<code> \$scope module uut \$end</code>	← 再進入 <code>uut</code> 這一層
<code> \$var wire 1 " clk \$end</code>	
<code> \$var wire 1 # reset \$end</code>	
<code> \$var reg 4 \$ count [3:0] \$end</code>	← <code>uut</code> 裡面的 <code>count</code> ，代號 <code>\$</code>
<code> \$upscope \$end</code>	← 退出 <code>uut</code>
<code>\$upscope \$end</code>	← 退出 <code>tb_counter</code>
<code>\$enddefinitions \$end</code>	← 目錄結束

這一段直接對應 GTKWave 左上角那棵樹：

```
▼ tb_counter      ← $scope module tb_counter
  uut              ← $scope module uut
```

每個訊號被指派一個單字元代號，這是為了省空間——後面幾百萬次的變化只要寫一個字元就好，不用每次寫 `tb_counter.count`。

第 24–30 行：時間 0

```
#0          ← 時間 = 0
$dumpvars
bx $        ← uut.count = xxxx (未知)
1#          ← reset = 1
0"          ← clk = 0
bx !        ← count = xxxx (未知)
$end
```

讀法（格式是「值」直接黏在「代號」前面）：

- `#0` = 「以下是時間 0 發生的事」
- `1#` = 把值 1 給代號 `#`（也就是 `reset`）→ `reset = 1`
- `0"` = `clk = 0`
- `bx !` = `b` 開頭代表多位元，`x` = 未知 → `count = xxxx`

這解釋了你圖上最左邊那一小段 X（紅色）——正反器剛通電，還沒被寫過任何值。

第 31–41 行：前兩個時脈

```
#5          ← 時間 5
b0 !        ← count = 0      ← 被 reset 清掉了
b0 $
1"          ← clk = 1        ← 第一個上升緣！

#10         ← 時間 10
0"          ← clk = 0

#15         ← 時間 15
b1 !        ← count = 1      ← 開始數了
b1 $
1"          ← clk = 1        ← 第二個上升緣
0#          ← reset = 0      ← 重置放開
```

對照 `counter.v`：在 `#5` 那個上升緣，`reset` 還是 1 → 走 `if (reset) → count <= 0`。這就是 `b0 !` 的來源。

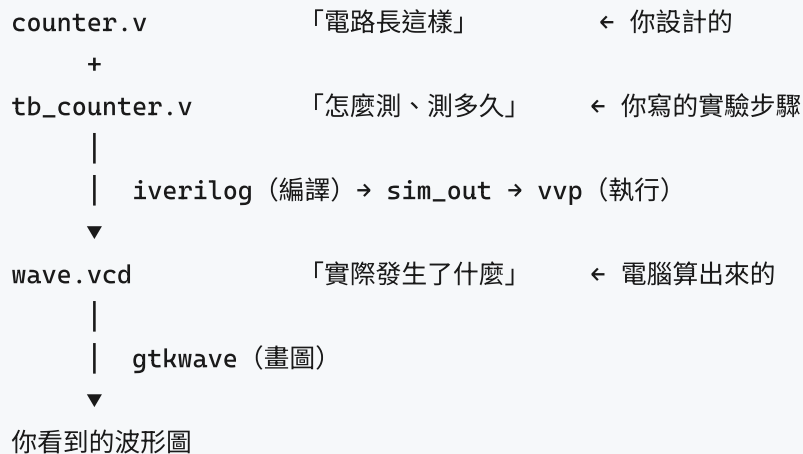
第 42 行之後：一路數上去

```
#25  b10  !    count = 2    (二進位 10)
#35  b11  !    count = 3    (二進位 11)
#45  b100 !    count = 4
#55  b101 !    count = 5
...
#115 b1011 !   count = 11   (二進位 1011)
```

規律：每 10 個時間單位（一個完整時脈週期）加 1，而且都發生在奇數（5、15、25...），也就是 `clk` 上升緣。

中間那些只有 `0` 的行（`#10`、`#20`、`#30`...）是 `clk` 掉下來，`count` 沒變——因為 `posedge` 只認上升緣。

4. 三個檔案跟那張圖的關係



逐項對照 — 圖上每個東西是哪裡來的

你在圖上看到	來自	哪一行
● clk 方波，週期 10	tb_counter.v	always #5 clk = ~clk;
● reset 前 15 是高的	tb_counter.v	reset = 1;
● reset 在 15 掉下來	tb_counter.v	#15 reset = 0;
● count 最前面是 X	正反器還沒被寫過	(沒有任何一行，是初始狀態)
● count 停在 0 那段	counter.v	if (reset) count <= 4'b0000;
● count 每次 +1	counter.v	else count <= count + 1;
● 只在 clk 上升時變	counter.v	always @(posedge clk)
圖的總長度 115	tb_counter.v	#15 + #100
圖存在這件事本身	tb_counter.v	\$dumpfile + \$dumpvars
左邊的樹 tb_counter ▶ uut	wave.vcd	\$scope module ...
時間軸單位是 sec	wave.vcd	\$timescale 1s
count 的每一個數字	wave.vcd	b0, b1, b10, ...

一句話總結

你在圖上看到的縱向（有哪些訊號、分幾層）來自 `wave.vcd` 的目錄段；橫向（什麼時候、發生什麼）由 `tb_counter.v` 決定「餵什麼」、`counter.v` 決定「怎麼反應」。

附註：一個你可能會問的細節

看 #15 那一刻，`reset` 掉成 0 和 `clk` 上升緣同時發生，而 `count` 卻變成了 1（不是 0）。

這是因為模擬器在同一個時間點先處理了 `reset = 0`，所以那個上升緣看到的是新值。這在業界叫 **race condition**（競態），是測試檔寫法造成的，不算錯但不嚴謹。正式的寫法會讓刺激訊號避開時脈邊緣（例如改在 `negedge` 時變動）。

[!TIP] 現在不用管它，知道有這回事就好——等你之後看到「為什麼結果跟我想的差一拍」，會回想起這裡。